

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa:

1. Memahami konsep *Transaction Control Language* (TCL) sebagai bagian dari SQL yang digunakan untuk mengatur dan mengendalikan proses transaksi pada basis data.
2. Memahami penggunaan perintah `SAVEPOINT` untuk membuat titik penyimpanan sementara dalam suatu transaksi.
3. Memahami penggunaan perintah `ROLLBACK` untuk membatalkan perubahan data dan mengembalikan kondisi basis data sebelum transaksi dijalankan.
4. Memahami penggunaan perintah `COMMIT` untuk menyimpan seluruh perubahan data secara permanen ke dalam basis data.

12.2 Dasar Teori

Transaction Control Language (TCL) merupakan bagian dari SQL yang berfungsi untuk mengatur dan mengendalikan proses transaksi pada basis data. Transaksi sendiri adalah sekumpulan perintah SQL yang dijalankan secara bersamaan sebagai satu kesatuan proses kerja. Melalui TCL, pengguna dapat menjaga konsistensi dan integritas data dengan menggunakan perintah pengendalian transaksi seperti `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`. TCL juga berperan penting dalam memastikan setiap transaksi dapat diproses dengan aman dan terstruktur di dalam sistem basis data serta mendukung mekanisme *read consistency* agar data tetap konsisten selama transaksi berlangsung (Avini *et al.*, 2025).

`SAVEPOINT` adalah perintah dalam *Transaction Control Language* (TCL) yang digunakan untuk membuat titik penyimpanan sementara di dalam suatu transaksi. Titik penyimpanan ini berfungsi sebagai penanda sehingga transaksi dapat dikembalikan ke kondisi tertentu tanpa memengaruhi seluruh proses transaksi

yang sedang berjalan. Dengan demikian, `SAVEPOINT` membantu pengguna dalam mengelola transaksi secara lebih terstruktur dan efisien (Oracle, 2026).

`ROLLBACK` adalah perintah yang digunakan untuk membatalkan atau mengembalikan perubahan data yang terjadi pada transaksi yang sedang berlangsung. Perintah ini memungkinkan basis data kembali ke kondisi sebelum transaksi dijalankan sehingga perubahan yang belum disimpan secara permanen tidak akan diterapkan. Selain itu, `ROLLBACK` juga dapat digunakan untuk mengembalikan transaksi ke titik tertentu melalui `SAVEPOINT` (Oracle, 2026).

`COMMIT` adalah perintah yang digunakan untuk mengakhiri transaksi dan menyimpan seluruh perubahan data secara permanen ke dalam basis data. Setelah perintah `COMMIT` dijalankan, perubahan yang dilakukan pada transaksi akan dapat dilihat oleh pengguna lain dan tidak dapat dibatalkan menggunakan `ROLLBACK`. Selain itu, perintah ini juga akan menghapus seluruh `SAVEPOINT` serta melepaskan *lock* atau penguncian data yang digunakan untuk menjaga agar data tetap konsisten selama proses transaksi berlangsung (Oracle, 2026).

12.3 Hasil dan Pembahasan

Berikut merupakan percobaan *query* berdasarkan modul *section* 18 yang telah dipelajari.

1. **Query:**

```
UPDATE copy_departments SET manager_id= 101  
WHERE department_id = 60;
```

Hasil :

```
SQL> UPDATE copy_departments SET manager_id= 101 WHERE department_id = 60;  
1 row updated.
```

Gambar 12.1 Mengubah `manager_id` pada tabel `copy_departments`

Penjelasan:

Query tersebut digunakan untuk mengubah data pada tabel `copy_departments`. Perintah `UPDATE` digunakan untuk mengganti nilai `manager_id` menjadi “101” pada data yang memiliki `department_id` = 60. Dengan *query* ini, departemen dengan ID “60” akan memiliki manager baru sesuai nilai yang dimasukkan.

2. **Query:**

```
SAVEPOINT one;
```

Hasil :

```
SQL> SAVEPOINT one;  
Savepoint created.
```

Gambar 12.2 Pembuatan `SAVEPOINT` setelah proses `UPDATE` pada tabel `copy_departments`

Penjelasan:

Query tersebut digunakan untuk membuat `SAVEPOINT` bernama “one” setelah proses `UPDATE` sebelumnya dijalankan. `SAVEPOINT` berfungsi sebagai titik penyimpanan sementara dalam transaksi sehingga perubahan data dapat dikembalikan ke kondisi tertentu menggunakan perintah `ROLLBACK`. Dengan adanya `SAVEPOINT one`, perubahan pada *query* sebelumnya masih bisa dibatalkan tanpa membatalkan seluruh transaksi.

3. *Query*:

```
INSERT INTO copy_departments(department_id,  
department_name, manager_id, location_id)  
VALUES(130, 'Estate Management', 102, 1500);
```

Hasil:

```
SQL> INSERT INTO copy_departments(department_id, department_name, manager_id, location_id) VALUES(130, 'Estate Management', 102, 1500);  
1 row created.
```

Gambar 12.3 Penambahan data baru ke tabel `copy_departments`

Penjelasan:

Query tersebut digunakan untuk menambahkan data baru ke dalam tabel `copy_departments` menggunakan perintah `INSERT INTO`. Data yang dimasukkan meliputi `department_id` "130", nama departemen "Estate Management", `manager_id` "102", dan `location_id` "1500". Dengan *query* ini, data departemen baru akan tersimpan ke dalam tabel `copy_departments`.

4. *Query*:

```
UPDATE copy_departments  
SET department_id = 140;
```

Hasil:

```
SQL> UPDATE copy_departments  
2 SET department_id = 140;  
  
10 rows updated.
```

Gambar 12.4 Mengubah seluruh nilai `department_id` pada tabel `copy_departments` menggunakan `UPDATE`

Penjelasan:

Query tersebut digunakan untuk mengubah nilai `department_id` pada tabel `copy_departments` menjadi "140" menggunakan perintah `UPDATE`. Karena *query* tidak menggunakan klausa `WHERE`, maka seluruh data pada kolom `department_id` di tabel akan berubah menjadi "140". Perintah ini perlu digunakan dengan hati-hati agar tidak mengubah seluruh data secara tidak sengaja.

5. *Query*:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:

```
SQL> SAVEPOINT one;
Savepoint created.

SQL> UPDATE copy_departments
  2 SET department_id = 140;

10 rows updated.

SQL> ROLLBACK TO SAVEPOINT one;
Rollback complete.
```

Gambar 11.5 Pengembalian transaksi ke `SAVEPOINT one` menggunakan `ROLLBACK`

Penjelasan:

Query tersebut digunakan untuk mengembalikan transaksi ke `SAVEPOINT one` yang telah dibuat sebelumnya. Dengan perintah `ROLLBACK TO SAVEPOINT`, perubahan data setelah `SAVEPOINT` dibuat akan dibatalkan, sedangkan perubahan sebelum `SAVEPOINT` tetap tersimpan. Perintah ini membantu pengguna membatalkan sebagian transaksi tanpa harus membatalkan seluruh proses.

6. *Query:*

```
COMMIT;
```

Hasil:

```
Savepoint created.  
SQL> UPDATE copy_departments  
2 SET department_id = 140;  
10 rows updated.  
SQL> ROLLBACK TO SAVEPOINT one;  
Rollback complete.  
SQL> COMMIT;  
Commit complete.
```

Gambar 12.6 Menyimpan transaksi secara permanen
menggunakan COMMIT

Penjelasan:

Query tersebut digunakan untuk menyimpan seluruh perubahan transaksi secara permanen ke dalam *database* menggunakan perintah COMMIT. Setelah COMMIT dijalankan, perubahan data yang sebelumnya dilakukan tidak dapat dibatalkan lagi dengan ROLLBACK. Perintah ini biasanya digunakan setelah proses INSERT, UPDATE, atau DELETE selesai dan data dianggap sudah benar.

12.4 Latihan

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query:

```
INSERT INTO mahasiswa (nim, nama, jurusan_id,  
prodi, angkatan) VALUES (2407110909, 'Kim  
Taehyung', 105, 'Teknik_Informatika', '2024');  
  
SAVEPOINT taehyung;  
  
DELETE FROM mahasiswa  
WHERE nama = 'Kim Taehyung';  
  
ROLLBACK TO SAVEPOINT taehyung;  
  
COMMIT;
```

Hasil:

```
SQL> INSERT INTO mahasiswa  
2 (nim, nama, jurusan_id, prodi, angkatan)  
3 VALUES  
4 (2407110909, 'Kim Taehyung', 105, 'Teknik_Informatika', '2024');  
  
1 row created.  
  
SQL> SAVEPOINT taehyung;  
  
Savepoint created.  
  
SQL> DELETE FROM mahasiswa  
2 WHERE nama = 'Kim Taehyung';  
  
1 row deleted.  
  
SQL> ROLLBACK TO SAVEPOINT taehyung;  
  
Rollback complete.  
  
SQL> COMMIT;  
  
Commit complete.
```

Gambar 12.7 Penggunaan perintah TCL untuk mengelola transaksi pada tabel mahasiswa

Penjelasan:

Query tersebut digunakan untuk menerapkan perintah TCL (Transaction Control Language) dalam mengelola transaksi pada tabel mahasiswa. Proses dimulai dengan menambahkan data mahasiswa “Kim Taehyung” ke dalam tabel, kemudian dibuat `SAVEPOINT taehyung` sebagai titik penyimpanan sementara transaksi. Setelah itu dilakukan proses `DELETE` untuk menghapus data tersebut, namun penghapusan dibatalkan menggunakan `ROLLBACK TO SAVEPOINT taehyung` sehingga data mahasiswa kembali seperti semula. Perintah `ROLLBACK` membantu pengguna membatalkan perubahan tertentu tanpa menghapus seluruh transaksi yang telah dilakukan. Terakhir, perintah `COMMIT` digunakan untuk menyimpan seluruh perubahan transaksi secara permanen ke dalam *database*.

12.5 Kesimpulan

Berdasarkan praktikum, dapat disimpulkan bahwa *Transaction Control Language* (TCL) merupakan bagian dari SQL yang digunakan untuk mengatur dan mengendalikan proses transaksi pada basis data. TCL berperan penting dalam menjaga konsistensi dan integritas data melalui pengelolaan transaksi secara terstruktur. Dengan adanya mekanisme transaksi, setiap perubahan data dapat diproses secara aman sehingga meminimalkan terjadinya kesalahan atau inkonsistensi data dalam sistem basis data.

Selain itu, penggunaan perintah `SAVEPOINT` memungkinkan pengguna membuat titik penyimpanan sementara di dalam transaksi. Dengan adanya `SAVEPOINT`, transaksi dapat dikembalikan ke kondisi tertentu tanpa harus membatalkan seluruh proses transaksi yang sedang berjalan. Hal ini membantu pengelolaan transaksi menjadi lebih fleksibel dan efisien, terutama ketika terjadi kesalahan pada sebagian proses transaksi.

Selanjutnya, perintah `ROLLBACK` digunakan untuk membatalkan perubahan data yang belum disimpan secara permanen sehingga basis data dapat dikembalikan ke kondisi sebelumnya. `ROLLBACK` juga dapat digunakan bersama `SAVEPOINT` untuk mengembalikan transaksi ke titik tertentu sesuai kebutuhan. Sementara itu, perintah `COMMIT` berfungsi untuk menyimpan seluruh perubahan data secara permanen ke dalam basis data. Setelah `COMMIT` dijalankan, perubahan data akan dapat dilihat oleh pengguna lain dan tidak dapat dibatalkan kembali menggunakan `ROLLBACK`. Dengan demikian, penggunaan `COMMIT`, `ROLLBACK`, dan `SAVEPOINT` sangat membantu dalam menjaga keamanan, konsistensi, dan kestabilan proses transaksi pada basis data.